Revisiting SLIM: Improved Learning Dynamics and Model Compactness in Symbolic Regression

Gorka Silva[1*], Lachlan Stewart[2*], Illya Bakurov[5], Mauro Castelli[4], Davide Farinati[4,6], Jose Manuel Muñoz Contreras[3], Leonardo Trujillio[3], and Leonardo Vanneschi[4]

[1] Universidad Complutense de Madrid, Spain
[2] Australian National University, Australia
[3] Tecnológico Nacional de México/Instituto Tecnológico de Tijuana, Mexico
[4] NOVA Information Management School (NOVA IMS), Universidade Nova de Lisboa, Campus de Campolide, Lisboa 1070-312, Portugal
[5] Michigan State University, USA
[6] Vita-Salute San Raffaele University, Milan, Italy Comprehensive Cancer Center/Unit of Urology; URI; IRCCS Ospedale San Raffaele, Milan, Italy

Abstract. Geometric Semantic Genetic Programming (GSGP) induces unimodal error surfaces for supervised learning problems, enabling efficient search within semantic space. However, its conventional operators cause rapid model growth, limiting interpretability. The Semantic Learning algorithm based on Inflate and deflate Mutations (SLIM) addresses this limitation through size-reducing deflate mutations, while preserving GSGP's theoretical properties. This work presents a systematic enhancement of SLIM through two sets of improvements: (i) theoretically grounded refinements, including optimal mutation step and linear scaling; and (ii) heuristic extensions, including Pareto-based tournament selection, multi-objective model identification, bounded mutation steps for implicit regularization and automatic algebraic simplification. A comprehensive evaluation on regression tasks demonstrates that all of the proposed SLIM enhancements, when considered separately, match or exceed baseline performance on test data, while achieving a significant reduction of model size. However, the best results were achieved when all enhancements are employed together. These results position the enhanced SLIM as a promising step toward more accurate, efficient, and interpretable symbolic regression for real-world data modeling.

Keywords: GP · Semantics · SLIM · Pareto Tournament · Linear Scaling · Optimal Mutation Step

1 Introduction

Many machine learning (ML) techniques have been proposed to derive data-driven regression models, including deep learning approaches [15] and state-of-the-art ensembles of decision trees [25]. However, standard ML approaches

[*] These authors contributed equally to this work

present an important trade-off: they achieve high accuracy but produce black-box models [1, 13]. While it is possible to derive useful explanations of how these models make inferences, achieving model interpretability is usually not feasible, given the size of the models or the way in which they are expressed [1].

When both model interpretability and accuracy are required, as is the case in Symbolic Regression (SR), Genetic Programming (GP) methods, such as Operon [3], GP-GOMEA [31] or PySR [5], among others, can be suitable approaches [12]. In fact, GP methods can potentially achieve this interpretability-accuracy balance by simultaneously optimizing model performance, structure and size. However, the syntactic search performed by GP can be inefficient and difficult to guide compared with conventional approaches used for ML training [12]. This happens because GP models are transformed through stochastic structural modifications (such as subtree crossover and mutation), that often result in abrupt or unpredictable changes in performance [21]. One way to overcome search inefficiency is to use geometric semantic operators (GSOs) [16]. These operators modify model syntax in such a way that the corresponding changes in the output (semantic) space are bounded. Such is the strategy of Geometric Semantic Genetic Programming (GSGP), which has been shown to achieve strong performance, thanks to its ability to induce a unimodal error surface for supervised learning problems [17]. The main issue with this method, however, is that the original mutation operator always produces large models (number of nodes in a syntax tree representation). While modeling accuracy is competitive, and GSGP can be implemented efficiently using parallel processing [27], it produces significantly larger models than other GP methods, thereby eliminating GP's core advantage compared to other ML techniques. However, a recent variant of GSGP called Semantic Learning algorithm based on Inflate and deflate Mutation (SLIM), has shown that it is possible to maintain the semantic properties while also using genetic operators that reduce model size. This variant has achieved good accuracy while dramatically reducing model size [28].

This work builds on SLIM and presents an enhanced version which accelerates convergence, increases accuracy and achieves a significant reduction in model size. To obtain this, several improvements are proposed. First, we incorporate simple enhancements that have already been shown to help GSGP in other works [18, 27]. In particular, by computing an optimal mutation step for each mutation event and performing linear scaling of the outputs of each model. Second, by incorporating other heuristics that have been used with other GP or ML systems, but not specifically with GSGP, such as Pareto tournament selection, Pareto-based selection of the best model found, and a bounded mutation step for regularization. These simple yet powerful enhancements to SLIM yield state-of-the-art performance among GSGP algorithms, moving it a step closer to becoming a leading symbolic regression framework by uniting strong predictive accuracy with the potential for improved interpretability.

The paper is organized as follows. Background is presented in Section 2, while our proposal is outlined in Section 3. Experimental work and results are discussed in Section 4, while conclusions and future work are outlined in Section 5.

2 Background on GSGP and SLIM

This section provides and overview of GSGP, focusing on Geometric Semantic Mutation (GSM). and outlines SLIM, its recently developed variant.

Geometric Semantic Genetic Programming. In supervised learning, GP individuals typically represent models that perform input-to-output mappings. For a GP individual P and an input set $X = \{x_1, x_2, \ldots, x_n\}$, we define the output vector $s_P = \{P(x_1), P(x_2), \ldots, P(x_n)\}$ as the *semantics* of individual P [16]. Classical GP employs search operators that randomly modify the syntactic structure of individuals. The semantic effects of these operators, however, are difficult to predict. Minor syntactic modifications may result in substantial semantic variations, degrading locality and hindering the identification of optimal solutions [20]. To overcome this limitation, semantic awareness has been incorporated into GP [29]. Among various approaches, GSGP is as a GP variant that substitutes conventional syntax-based operators with Geometric Semantic Operator (GSO)s [16]. GSOs, despite altering individual syntax, have predictable geometric properties within the semantic space and induce a unimodal error surface. For a parent $T : \mathbb{R}^n \to \mathbb{R}$, GSM produces the function $T_M = T + \mathrm{ms} \cdot (T_{R1} - T_{R2})$, where ms is a parameter called mutation step and T_{R1} and T_{R2} denote random real functions with outputs constrained to the interval $[0, 1]$. This constraint is typically imposed by wrapping the two random expressions within a sigmoid function. GSM produces an individual located within a hyper-sphere of radius ms centered at the parent's semantics within semantic space.

Semantic Learning algorithm based on Inflate and deflate Mutation. While several variants of GSGP have been proposed [4, 19, 23], it has one important limitation. The offspring produced by GSOs are always larger than their parents, leading to larger models with poor interpretability [16]. Various solutions have been developed to mitigate this problem, including dynamic population strategies [7] and gradient descent integration [23]. Nevertheless, most current methods still rely on the GSO developed by [16] or employ new operators that generate offspring exceeding parent size [2]. Recently, a novel GSM was developed [28] that preserves the characteristics of conventional semantic mutation (now termed Inflate Geometric Semantic Mutation (IGSM)), while simultaneously generating individuals smaller than their parents. This novel operator, designated Deflate Geometric Semantic Mutation (DGSM), owes its introduction to two intuitions. First, the IGSM definition from the previous section can trivially be reformulated as:

$$IGSM\,(T) = T + \mathrm{ms} \cdot (T_{R1} - T_{R2}) = T - \mathrm{ms} \cdot (T_{R2} - T_{R1})\,,$$

Second, the two random programs, T_{R1} and T_{R2}, are exchangeable as they constitute independent random variables sharing identical probability distributions. Consequently,the definition of IGSM may be rewritten as:

$$IGSM\,(T) = T - \mathrm{ms} \cdot (T_{R1} - T_{R2})\,.$$

Consider now an individual T that has undergone IGSM application, say, three times, yielding the new individual:

$$T + \mathrm{ms} \cdot (T_{R1} - T_{R2}) + \mathrm{ms} \cdot (T_{R3} - T_{R4}) + \mathrm{ms} \cdot (T_{R5} - T_{R6})\,.$$

Because the two definitions are equivalent, we may now apply IGSM again employing subtraction, producing the individual:

$$T + \mathrm{ms} \cdot (T_{R1} - T_{R2}) + \mathrm{ms} \cdot (T_{R3} - T_{R4}) + \mathrm{ms} \cdot (T_{R5} - T_{R6}) - \mathrm{ms} \cdot (T_{R7} - T_{R8})\,.$$

Individual size continues increasing with each IGSM application. However, by applying IGSM with subtraction and reusing a random program pair from a prior mutation event, such as for instance T_{R3} and T_{R4}, we can simplify the expression to:

$$T + \mathrm{ms} \cdot (T_{R1} - T_{R2}) \cancel{+\mathrm{ms} \cdot (T_{R3} - T_{R4})} + \mathrm{ms} \cdot (T_{R5} - T_{R6}) \cancel{-\mathrm{ms} \cdot (T_{R3} - T_{R4})}\,.$$

The resulting individual possesses a smaller genotype compared to its parent. This motivates the DGSM, which simply removes a previously added term. DGSM maintains the same semantic characteristics as IGSM, perturbing each semantic coordinate within $[-\mathrm{ms}, \mathrm{ms}]$, and inducing a unimodal error surface for supervised learning problems, exactly as IGSM.

The SLIM algorithm, as its name indicates, utilizes both IGSM and DGSM as two independent operators, each having its own application probability, with the sole constraint that when a program contains only one term, solely IGSM may be applied. Specifically, the application frequency of each GSM is governed by a parameter that regulates the probability of applying DGSM rather than IGSM. Setting this parameter to zero results in an exclusive IGSM application, rendering SLIM entirely equivalent to conventional GSGP.

A common criticism directed at SLIM, and in particular at the DGSM, concerns the fact that, during the evolutionary process, parts of the genotype that were previously accepted may later be removed. At first glance, this behavior appears counterintuitive: SLIM operates on a unimodal fitness landscape, and genetic material that was added and accepted earlier during evolution should have contributed to moving the solution closer to the target. Removing such code should always push the solution further away. However, this intuition is wrong. At each generation, new pieces of code are introduced, and components that were once beneficial may become detrimental in the presence of newly added genotypic elements. This phenomenon is addressed and empirically demonstrated by Farinati et al. [8].

Variants of SLIM. In [22], the authors examined three distinct methods (SIG2, ABS, SIG1) for defining a function that produces values within the range [-ms, ms], along with two approaches for perturbing an individual to accomplish ball mutation in the semantic space. The three functions that produce values within [-ms, ms] are:

1. Utilizing two random trees T_{R1} and T_{R2}, each passed through the sigmoid function S (mapping outputs to $[0,1]$) as follows (standard GSM):

$$\mathrm{SIG2} = ms \cdot (S(T_{R1}) - S(T_{R2}))\,.$$

2. Employing a single random tree T_R normalized using the absolute value:

$$\mathrm{ABS} = ms \cdot (1 - 2/(1 + |T_R|))\,.$$

3. Utilizing a single random tree T_R passed through the sigmoid function S:

$$\mathrm{SIG1} = ms \cdot (2 \cdot S(T_R) - 1)\,.$$

Also, a small perturbation can be accomplished by adding a positive or negative quantity close to zero, or by multiplying by a factor slightly smaller or larger than one. Correspondingly, there are two methods to mutate individuals in SLIM: (1) IGSM adds the function, while DGSM subtracts it; and (2) IGSM multiplies by $(1 + \text{function})$, while DGSM divides by it.

Combining both methods for perturbing an individual with the three functions defined for mutation, we obtain six SLIM variants. These are designated as SLIM+ or SLIM$*$, depending on whether they employ addition or multiplication, followed by SIG2, ABS, or SIG1, based on the function applied. This work focuses on SLIM+ABS due to its strong performance, clearly demonstrated in previous work [22], and its simple construction.

Linked-List Implementation of SLIM. In order to implement SLIM, it is useful to represent each individual's genotype as a linked list of subexpressions [22]. The IGSM appends one element to the list's end, whereas the DGSM removes one element from the list. Depending on the variant employed, the individual's semantics can be computed by summing or multiplying the semantics of all blocks. A visual representation of the linked list implementation and the application of the IGSM and DGSM are reported in Figure 1.

An open-source implementation[7] of SLIM was introduced in [24], and it was used in this study. The code for the full reproducibility of the experiments ran in this work can be found at `https://github.com/lachlanjs/species-ss-slim-gsgp`.

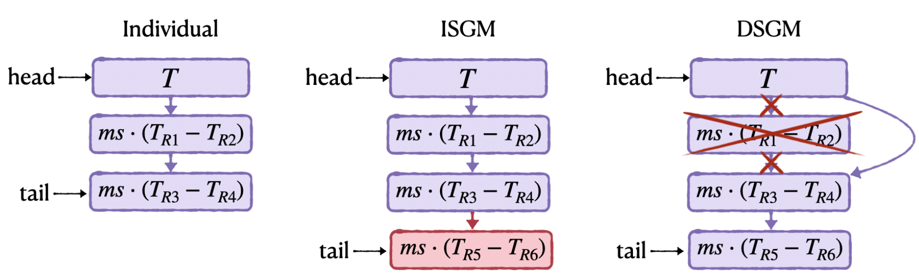


Fig. 1: An example genotype of a SLIM individual, with a visual representation of the effect of the IGSM and DGSM on its genotype. IGSM appends an item at the end of the list, while DGSM removes an item.

[7] https://github.com/DALabNOVA/slim

3 Proposed extensions to SLIM

Initial results with SLIM have been promising [8, 24, 28, 30]. However, this work hypothesizes that performance can still be improved by enhancing the search process. First, to improve the predictive accuracy of the models. Second, to reduce model size and increase the potential for interpretability.

3.1 Improving model accuracy

The first two algorithmic enhancements are expected to improve performance. Standard GSGP and SLIM use mutation steps ms that are randomly generated at each mutation event, as suggested in [9]. However, the ms need not be be random. Given the unimodal fitness landscape induced by the GSGP operators [16], the mutation step can be optimally computed based on the parent, the target, and the random programs involved. On the other hand, it is trivial to adjust the scale and the bias of a model's output to better fit the target variable, using a simple adjustment known as linear scaling [10, 18]. Both approaches have been shown to improve the accuracy of GSGP, and should have a positive effect on SLIM performance. Moreover, these heuristics improve the convergence of the algorithm, allowing it to achieve low errors with fewer iterations. This can also have a positive effect on model size, by reducing the amount of inflate mutations.

Optimal Mutation Steps with regularization. A key hyperparameter of IGSM is the mutation step ms, which can be statically or randomly set at each generation or mutation event. Another strategy is to compute an optimal mutation step (OMS) at each mutation event [9, 14]. Let $\mathbf{s}$ be the semantic vector of program T, $\mathbf{t}$ the desired or target semantic vector, and $\mathbf{s}_R$ the semantic vector produced by the random subtree added during mutation, in particular considering, for example, the +SIG or +ABS variants. Then, the goal is to find a program T such that

$$\mathbf{t} = \mathbf{s} + \mathrm{ms} * \mathbf{s}_R \ . \tag{1}$$

For any $\mathbf{s}$ and $\mathbf{s}_R$, then we can try to find the best possible mutation step, by

$$\mathrm{ms} = (\mathbf{t} - \mathbf{s}) \cdot \mathbf{s}_R^{-1}, \tag{2}$$

where $\mathbf{s}_R^{-1}$ is the pseudo-inverse of $\mathbf{s}_R$, which can be efficiently computed using the Moore-Penrose method [9]. One drawback is that under certain conditions, the pseudo-inverse can become unstable, producing either very large or very small values. Therefore, we include a simple regularization step to the OMS (ROMS) by bounding the values from above and from below, such that the ms is clipped to ms_l (-ms_l) when it goes beyond this limit. Moreover, it is set to 0 if it falls below a certain threshold $\mathrm{ms} < \mathrm{ms}_\epsilon$. Practically, this means that the mutation event is canceled when it has a negligible effect on semantics.

Linear Scaling (LS). It modifies the fitness function by calculating the slope and intercept of each GP model [10, 18], such that the sum of the squared errors between the target and the model outputs is minimized. The slope and intercept are computed on the training data, and used to make predictions at inference time. By computing the slope and intercept of the data, GP can search for the model that best fits the target shape without accounting for general trends.

3.2 Reducing model size

The main limitation of GSGP is its tendency to produce large models. While the deflate mutation helps addresses this issue, this work proposes additional heuristics to curtail model growth. The ROMS should already have a positive effect on model size when it sets ms to 0, effectively cancelling some mutation events. Several other improvements are presented below.

Pareto selection. Pareto criteria are used for parent selection and for choosing the best model at the end of a run. Pareto dominance, in terms of model size and accuracy, has been used to reduce code bloat in GP before [6, 11, 26], but not with GSGP. Pareto tournaments (PT) are for parent selection [11, 26]. Like in regular tournament selection, a PT takes as input n randomly chosen individuals from the population. Afterward, the non-dominated individuals are identified, and one is randomly chosen as the tournament winner. Solutions are evaluated based on error (fitness) and size (number of nodes). Moreover, to select the final model to return as the best solution found, three steps are performed. First, the set of non-dominated solutions in the final population is identified. Second, we eliminate outlier individuals using the standard $1.5 \times IQR$ rule, considering either fitness or model size. Finally, the solution in the non-dominated set that is closest to the ideal solution (the origin of objective space) is computed, referred to as the optimal compromise model. The method also returns the model with the best fitness (lowest error) and the best (smallest) size.

Automatic model Simplification (AS). Given the way that GP evolves models, relying on random syntactic modifications, it is desirable to simplify the models found, which is feasible because they are expressed symbolically. Therefore, before choosing the best solution from the final population, all non-dominated models are simplified. Each SLIM individual is first converted into a tree structure in PyTorch, and three categories of simplification rules are applied. First, identity simplifications (e.g.,$x + 0 \rightarrow x$, $x \times 1 \rightarrow x$, $x - x \rightarrow 0$), constant folding (e.g., $5.0 + 3.0 \rightarrow 8.0$), and nested constant optimization (e.g., $c_1 \times (c_2 \times x) \rightarrow (c_1 \times c_2) \times x$). The process is performed recursively via bottom-up traversal. However, given the type of mutations used with SLIM (ABS or SIG), there is a limit to the amount of viable simplifications that can be made given their use of absolute value or exponential functions [2]. Nonetheless, given the use of a regularized OMS, where some values are set to zero, at least in those cases, model simplification is expected to have a non-negligable impact on model size.

4 Experimental Study

The experimental study evaluates the combined effect of the proposed enhancements to SLIM and the impact of each one individually. For this, 14 regression problems that were used to evaluate SLIM in previous work are used [28, 30]. The domain, number of samples and number of features of the employed datasets are reported in Table 1. For evaluation, the algorithm is executed 30 times on each dataset, using 60% for training, 20% for validation and 20% for testing. From each run, three models are chosen, based on training performance, from the final dominated set in the final population (for all variants, including the baseline): (i) the model with the best fitness (BF); (ii) the one with the opti-

Table 1: Summary of datasets used for experimental evaluation. S: Number of samples. F number of features

ID	Dataset	S	F	ID	Dataset	S	F
1	Airfoil	1502	5	8	Diabetes	442	10
2	Bike_sharing	730	14	9	Efficiency_cooling	768	8
3	Bioavailability	359	241	10	Efficiency_heating	768	8
4	Boston	506	13	11	Forest_fires	517	12
5	Breast_cancer	569	30	12	Parkinson_updrs	5875	19
6	Concrete_slump	103	7	13	PPB	131	626
7	Concrete_strength	1030	8	14	Resid_build_sale_price	1460	79

e

mal compromise between size and fitness (OC); and (iii) the one with the best (smallest) size (BS).

To quantify the impact of each proposed improvement to SLIM, we analyzed nine new variants that integrate the proposed extensions, namely: optimal mutation step (OMS), Linear Scaling (LS), Pareto Tournament (PT), and automatic simplification (AS), using the following naming convention:

- First, we consider the baseline SLIM using +ABS inflate mutation, denoted as “BASE + .
- Second, we study the “ALL" variant, that uses all of the proposed extensions, joined together in a unique algorithm.
- Third, we evaluate the impact of adding single improvements to the baseline method, denoted as “BASE + [PT | OMS | LS | AS]", to indicate which component was used; these are four variants.
- Finally, we consider four further variants, removing one of the extensions from the “ALL" method, denoted as “ALL - [PT | OMS | LS | AS]" to indicate which component was not used.

The experiments evaluate the effect produced by each enhancement, but also perform an ablation study to assess what happens when each one is omitted. Results are presented in four ways, considering the median over all 30 runs of the test Root Mean Squared Error (RMSE) and the size of the model (number of nodes): (1) plots that compare the proposed variants with the baseline SLIM implementation, to report the reduction in test RMSE and size as a percentage of the SLIM baseline; (2) median performance of the BF, BS and OC model found by each variant, shown in numerical tables; (3) critical difference diagrams showing the statistical significance of the observed results, separated by the final choice of model; (4) critical difference diagrams, which compare each final model choice for the ALL variant and the BF model found by SLIM. Table 2 reports the used hyperparameters, based on those found in previous works with SLIM [28]. The function set uses arithmetic operators with protected division.

4.1 Results and Discussion

Figure 2 summarizes the comparative results, relative to the baseline SLIM (BASE). Figure 2(a) shows a curve for each variant, considering each of the three

Table 2: Parameter settings.

Param.	Value	Param.	Value	Param.	Value	Param.	Value
Gens.	100	p(inflate)	0.5	p(cross)	0.0	Train %	60
Reps.	30	ms_l	100.0	Binary Tourn. size	2	Val %	20
|pop|	100	ms_ϵ	0.1	Pareto Tourn. size	5	Test %	20

models chosen from the non-dominated set of solutions in the final population (BF, BS and OC). The curves show the change in median performance (RMSE on the left and size on the right) achieved on each problem, while the performance of BASE is a horizontal line at zero. Negative values indicate a reduction in error or size, which are the desired outcomes. Figure 2(b) summarizes the average (over all problems) of the median (over all runs) change in performance, compared to the BF model found with BASE; this is the model normally returned by GSGP and SLIM. Again, lower values (left-button corner) are preferable in both axis. A line connect the three models (BF,BS and OC) found by each variant.

A statistical comparison is presented in Figure 3 based on the average rank (considering median performance) of each variant, shown as critical difference diagrams. These are based on the Friedman multigroup test and the Wilcoxon signed-rank test for pairwise comparisons with an $\alpha = 0.05$ and p-value correction using the Holm method. Horizontal lines indicate that there is no statistically significant difference between a subset of connected variants. Table 3 presents the median performance results of all variants based on test RMSE, while Table 4 shows the same for model size, for all models (BF, BS and OC).

The results show that the proposed extensions provide significant improvements over canonical SLIM, for both predictive accuracy and model size. In particular, the test RMSE shows a marked reduction when either OMS or LS are applied. Importantly, the regularized version of OMS (ROMS) does not negatively affect performance. This suggests that omitting small, semantically insignificant, moves can help produce more compact models without sacrificing accuracy. The combination of OMS and LS does not degrade performance in any of the studied test problems. On the contrary, very strong results are typically obtained when both extensions are used together. Conversely, when either OMS or LS is removed, accuracy can slightly decrease; however, models that include one or both extensions consistently achieve a lower test RMSE than the baseline. Furthermore, these extensions tend to slightly reduce model size, indicating that these techniques can provide an additional regularizing effect.

The proposed extensions have a pronounced impact on model size, particularly when the PT selection is employed. The baseline SLIM model already generates more compact models when PT is added, but the smallest models are obtained when PT is combined with the other proposed enhancements. It is worth noting, however, that PT alone tends to negatively affect predictive accuracy. In terms of RMSE, the highest-performing variant is ALL-PT, whereas the BASE+PT configuration ranks among the lowest. The use of PT appears to be beneficial when it is combined with other enhancements, since several variants

Table 3: Median fitness results. Rows for each dataset are ordered: Best Fitness, Best Size, Optimal Compromise. Values in bold, underline, and green represent the best in row, dataset and improvement over SLIM respectively.

Prob.	BASE	B. + PT	B. + OMS	B. + LS	B. + AS	A. - PT	A. - OMS	A. - LS	A. - AS	ALL
	(57.01)	58.15	5.03	5.55	57.01	4.94	5.59	5.39	4.87	**4.87**
1	59.18	64.76	6.69	5.62	59.18	**5.49**	5.71	66.14	5.71	5.71
	57.53	60.57	5.27	5.56	57.53	**5.00**	5.62	16.14	5.03	5.03
	(1035.19)	1036.01	592.57	583.86	1035.19	**375.87**	584.29	603.32	385.04	389.37
2	1037.27	1041.48	664.97	584.57	1037.25	**444.81**	585.31	1041.97	585.68	585.68
	1036.19	1039.02	618.45	584.17	1036.19	**380.70**	584.55	759.09	452.93	456.54
	(46.81)	47.42	31.54	31.22	46.81	31.38	31.18	**30.05**	31.37	31.46
3	48.26	50.35	31.74	30.89	48.11	**30.65**	30.97	54.80	30.74	30.97
	47.40	48.68	**30.61**	31.05	47.40	30.61	31.07	31.88	31.23	31.33
	(9.27)	9.67	5.18	5.33	9.27	4.81	5.51	5.60	4.80	**4.80**
4	9.70	10.79	7.59	5.66	9.70	**5.47**	5.84	10.79	5.84	5.84
	9.43	10.09	5.79	5.40	9.43	**4.86**	5.61	6.47	5.07	5.07
	(0.31)	0.34	0.27	**0.26**	0.31	0.26	0.26	0.27	0.27	0.26
5	0.47	0.54	0.32	0.28	0.47	**0.27**	0.31	0.55	0.31	0.31
	0.36	0.38	0.28	**0.26**	0.36	0.26	0.27	0.31	0.28	0.28
	(9.56)	9.57	7.77	8.20	9.56	8.28	7.97	**7.57**	7.70	8.16
6	9.97	11.70	8.78	8.06	9.97	8.33	7.95	11.70	7.95	**7.95**
	9.72	10.26	**7.70**	7.99	9.72	8.15	7.92	8.35	7.95	8.11
	(22.81)	23.56	**8.66**	8.99	22.81	8.94	10.87	9.42	9.64	9.69
7	24.97	28.38	12.33	11.62	24.97	**10.43**	13.55	29.06	13.63	13.63
	23.58	25.70	9.64	9.58	23.58	**9.26**	12.29	13.90	10.60	10.76
	(154.05)	154.94	69.61	58.19	154.05	**56.84**	57.99	71.43	57.40	57.20
8	155.87	159.64	79.12	**58.93**	155.85	60.38	61.39	158.90	61.39	61.39
8	154.71	157.26	71.01	58.16	154.71	57.80	59.05	78.94	57.68	**57.66**
	(5.75)	6.59	3.53	3.46	5.75	**3.29**	3.48	3.66	3.30	3.30
9	6.50	10.23	4.29	3.58	6.50	**3.51**	3.88	10.38	3.84	3.84
	5.98	7.63	3.69	3.54	5.98	**3.33**	3.61	4.58	3.48	3.47
	(6.08)	6.33	3.32	3.12	6.08	**2.98**	3.56	3.42	3.09	3.08
10	6.29	8.97	4.00	3.47	6.29	**3.15**	4.16	9.79	4.11	4.11
	6.15	7.04	3.54	3.22	6.15	**3.00**	3.74	4.96	3.39	3.33
	(1.39)	1.39	1.39	1.41	1.39	**1.38**	1.40	1.40	1.39	1.39
11	1.41	1.38	1.41	1.39	1.41	1.39	1.39	**1.38**	1.39	1.39
	1.40	1.40	1.39	1.41	1.40	**1.38**	1.39	1.39	1.39	1.39
	(11.68)	11.82	10.07	10.03	11.68	**9.89**	10.05	10.23	9.90	9.90
12	11.92	12.77	10.43	10.06	11.92	**10.02**	10.14	14.03	10.14	10.14
	11.81	12.14	10.17	10.04	11.81	**9.91**	10.08	10.43	10.00	9.99
	(36.87)	37.17	**29.59**	30.93	36.87	30.69	29.80	30.05	31.83	32.00
13	37.09	38.88	33.30	**30.20**	37.09	30.75	30.89	39.65	31.24	31.16
	36.98	37.73	**29.94**	30.21	36.98	30.40	30.08	30.80	31.32	31.67
	(106.15)	106.24	66.76	80.05	106.15	65.74	81.09	71.22	60.20	**60.20**
14	107.61	110.88	77.43	81.02	107.61	**66.44**	86.05	110.65	83.33	82.37
	106.89	108.29	69.20	80.98	106.89	65.06	81.81	79.80	63.68	**63.68**

that apply such a combination tend to achieve a low RMSE, comparable to the best performing variants, while maintaining substantially smaller models. The results achieved through PT are consistent with our intuition: by introducing model size as a secondary objective, the evolutionary search is biased away from focusing exclusively on fitness, thereby evolving more compact models.

The AS extension shows a similar, but less pronounced, effect to PT, with a consistent yet modest reduction in model size. The fact that the effect of AS on model size is modest is due to the difficulty of automatically simplifying functions that are highly non-linear, like the ones added by the IGSM. However, AS leads to a more notable reduction in size when paired with OMS, since some IGSM events are removed during the simplification step when the ms is set to zero.

Table 4: Median size results. Rows for each dataset are ordered: Best Fitness, Best Size, Optimal Compromise. Values in bold, underline, and green represent the best in row, dataset and improvement over SLIM respectively.

Prob.	BASE	B. + PT	B. + OMS	B. + LS	B. + AS	A. - PT	A. - OMS	A. - LS	A. - AS	ALL
1	(58.00)	57.00	68.00	65.00	55.00	58.00	54.00	**50.00**	53.00	53.00
	36.00	3.00	35.00	29.00	34.00	29.00	7.00	**<u>3.00</u>**	6.00	6.00
	45.00	29.00	44.00	45.00	42.00	37.00	27.00	**11.00**	21.00	20.00
2	(49.00)	49.00	51.00	51.00	47.00	47.00	49.00	48.00	45.00	**43.00**
	31.00	4.00	35.00	28.00	31.00	27.00	4.00	3.00	3.00	**<u>3.00</u>**
	38.00	25.00	41.00	37.00	37.00	33.00	23.00	21.00	19.00	**19.00**
3	(40.00)	39.00	39.00	40.00	39.00	39.00	39.00	**39.00**	42.00	42.00
	27.00	3.00	27.00	27.00	27.00	27.00	3.00	3.00	3.00	**<u>3.00</u>**
	31.00	21.00	31.00	33.00	31.00	31.00	21.00	**9.00**	19.00	20.00
4	(49.00)	47.00	47.00	48.00	48.00	45.00	45.00	45.00	44.00	**43.00**
	33.00	3.00	27.00	28.00	33.00	23.00	3.00	3.00	3.00	**<u>3.00</u>**
	41.00	23.00	33.00	35.00	39.00	31.00	18.00	**14.00**	17.00	17.00
5	(39.00)	**26.00**	44.00	45.00	39.00	42.00	41.00	39.00	41.00	40.00
	15.00	3.00	27.00	25.00	13.00	26.00	3.00	3.00	3.00	**<u>3.00</u>**
	25.00	11.00	31.00	31.00	23.00	31.00	15.00	**11.00**	15.00	14.00
6	(57.00)	51.00	55.00	56.00	53.00	57.00	53.00	53.00	51.00	**48.00**
	31.00	3.00	31.00	30.00	31.00	27.00	6.00	3.00	4.00	**<u>3.00</u>**
	41.00	23.00	39.00	39.00	39.00	36.00	27.00	**11.00**	17.00	21.00
7	(52.00)	49.00	53.00	49.00	51.00	49.00	52.00	48.00	48.00	**48.00**
	35.00	3.00	29.00	27.00	35.00	27.00	5.00	**<u>3.00</u>**	5.00	5.00
	43.00	26.00	35.00	35.00	41.00	36.00	24.00	**11.00**	21.00	19.00
8	(60.00)	54.00	55.00	44.00	57.00	44.00	44.00	54.00	38.00	**36.00**
	39.00	6.00	32.00	23.00	37.00	15.00	3.00	5.00	3.00	**<u>3.00</u>**
	48.00	33.00	41.00	32.00	43.00	26.00	19.00	**11.00**	17.00	16.00
9	(56.00)	55.00	49.00	56.00	53.00	50.00	47.00	51.00	47.00	**47.00**
	35.00	5.00	29.00	31.00	35.00	29.00	**<u>3.00</u>**	5.00	5.00	5.00
	45.00	25.00	35.00	39.00	43.00	37.00	19.00	**11.00**	18.00	19.00
10	(53.00)	53.00	52.00	53.00	52.00	52.00	**47.00**	51.00	53.00	49.00
	35.00	5.00	30.00	29.00	34.00	30.00	4.00	**<u>4.00</u>**	5.00	5.00
	43.00	26.00	37.00	37.00	41.00	38.00	20.00	**14.00**	23.00	23.00
11	(41.00)	**37.00**	43.00	43.00	40.00	43.00	42.00	43.00	42.00	41.00
	18.00	3.00	28.00	25.00	16.00	27.00	3.00	3.00	3.00	**<u>3.00</u>**
	27.00	**16.00**	35.00	32.00	27.00	34.00	17.00	20.00	19.00	19.00
12	(51.00)	46.00	46.00	46.00	49.00	45.00	48.00	45.00	46.00	**45.00**
	31.00	3.00	27.00	29.00	31.00	27.00	3.00	3.00	3.00	**<u>3.00</u>**
	37.00	21.00	33.00	37.00	37.00	33.00	21.00	**11.00**	17.00	17.00
13	(39.00)	41.00	39.00	39.00	39.00	39.00	39.00	39.00	39.00	**39.00**
	31.00	3.00	27.00	27.00	29.00	27.00	3.00	3.00	3.00	**<u>3.00</u>**
	35.00	23.00	33.00	31.00	34.00	31.00	19.00	**15.00**	19.00	19.00
14	(39.00)	40.00	39.00	39.00	39.00	39.00	39.00	39.00	37.00	**36.00**
	27.00	3.00	27.00	27.00	27.00	24.00	3.00	3.00	3.00	**<u>3.00</u>**
	35.00	22.00	31.00	31.00	35.00	30.00	15.00	**11.00**	12.00	13.00

In general, the proposed extensions to SLIM have a clear impact on performance and model size. But results also depend on the final model selection. While performance is clearly improved when considering the BF model, the improvements in size are less notable in that case compared to the baseline.

Finally, a comparison between the baseline SLIM and the ALL variant is presented, the variant that includes all of the proposed extensions. Analysis focuses on this variant because it achieves the best performance-size tradeoff (see Figure 2(b)), and also because it is the simpler one to use by practitioners, applying all heuristics improvements instead of toggling them on or off. Figure 4(a) compares the median test RMSE, shown as a critical difference diagram, while Figure 4(b) shows the same for median size. In all cases, the combined extensions clearly, and substantially, outperform the baseline SLIM. Moreover, in terms of test RMSE, even the BS model found by the ALL variant achieves competitive performance. While the average rank aligns with intuition (BF achieves the best test RMSE, followed by OC and BS), the statistical test suggests that using the smallest model can be a good strategy. This is a relevant achievement, because this means that it is feasible to choose the smallest BS, and so potentially the most interpretable one, without a substantial loss in performance.

5 Conclusions and Future Work

This study addressed a fundamental challenge for Symbolic Regression (SR): achieving strong predictive performance while maintaining compact solutions. We presented a systematic enhancement of the Semantic Learning algorithm based on Inflate and Deflate Mutations (SLIM), a recent variant of Geometric Semantic Genetic Programming (GSGP) that combines the theoretical advantages of semantic search with mechanisms that actively control bloat.

Two complementary sets of modifications have been proposed. The first set incorporated algorithmic refinements with established theoretical foundations: (i) optimal mutation step computation with regularization bounds to ensure numerical stability, and (ii) linear scaling of model outputs. The second set introduced more direct extensions: (i) Pareto-based tournament selection, (ii) end-of-run Pareto-optimal model identification, and (iii) automatic algebraic simplification of candidate models before final selection.

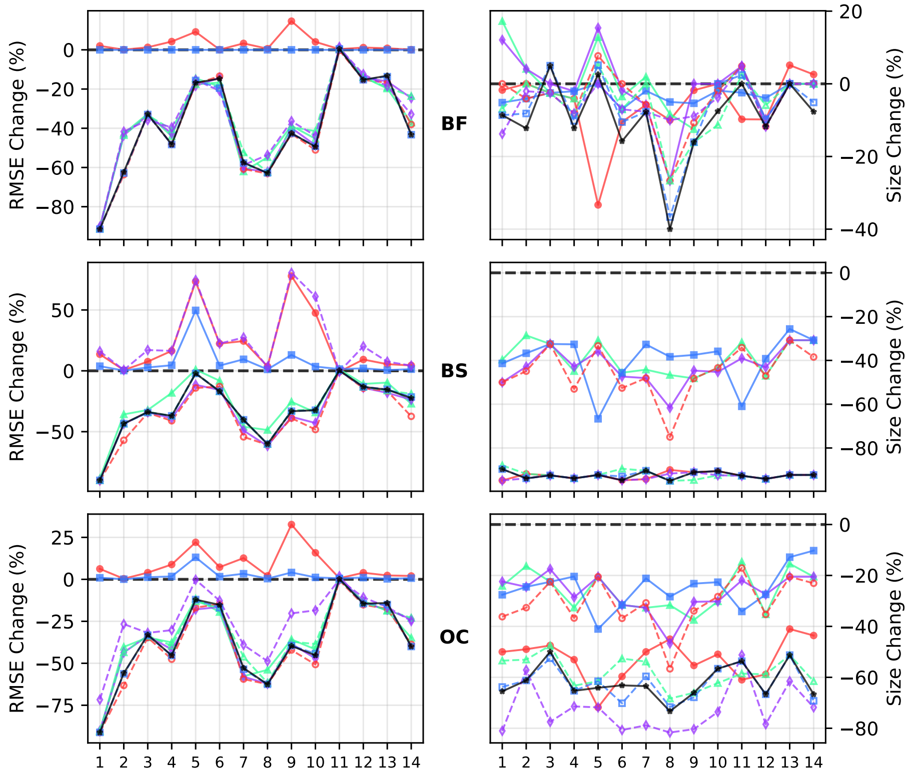
Empirical validation demonstrated that the proposed extensions work synergistically to improve the predictive accuracy and compactness of SR models generated by the algorithm. The improvements are consistent across 14 test datasets, always matching or exceeding the performance of SLIM.

In future work, it would be worthwhile to study the effects of these extensions in more detail. We conjecture that our extensions (particularly those that encourage compact solutions) may cause a moderate regulatory effect, thus contributing to preventing overfitting. Additional theoretical analyses of the convergence properties and semantic landscape induced by the proposed methods may also provide further insight into the mechanisms underlying SLIM's success.

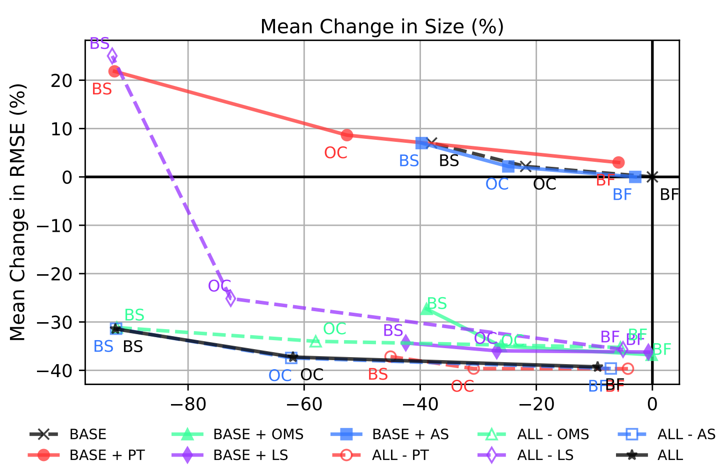
Another promising direction for future research involves the application of the improved SLIM to diverse domains, including scientific discovery and data-driven modeling in engineering and personalized medicine. Finally, it will be important to measure, in some way, if the improvements in model size are actually leading to increased interpretability in real-world scenarios.

Acknowledgments. Thanks to the SPECIES Society and the 2025 SPECIES Summer School Coordinator Anna Esparcia-Alcázar for nurturing an invaluable space where scientific ideas can flourish. This work was supported by national funds through FCT (Fundação para a Ciência e a Tecnologia), under the project - UID/04152/2025 - Centro de Investigação em Gestão de Informação (MagIC)/NOVA IMS[8] - (2025-01-01/2028-12-31) and UID/PRR/04152/2025[9] (2025-01-01/ 2026-06-30). This work was supported by TecNM (Mexico) project 19400.24-P and SECIHTI (Mexico) project CF-2023-I-724.

[8] https://doi.org/10.54499/UID/04152/2025
[9] https://doi.org/10.54499/UID/PRR/04152/2025



(a)



(b)

Fig. 2: (a): Percentage change of the median test RMSE and size relative to the baseline for three models: Best Fitness (BF), Best Size (BS) and Optimal Compromise (OC). (b): Average median percentage change of the same values, calculated over the fourteen datasets. In all plots lower values are better

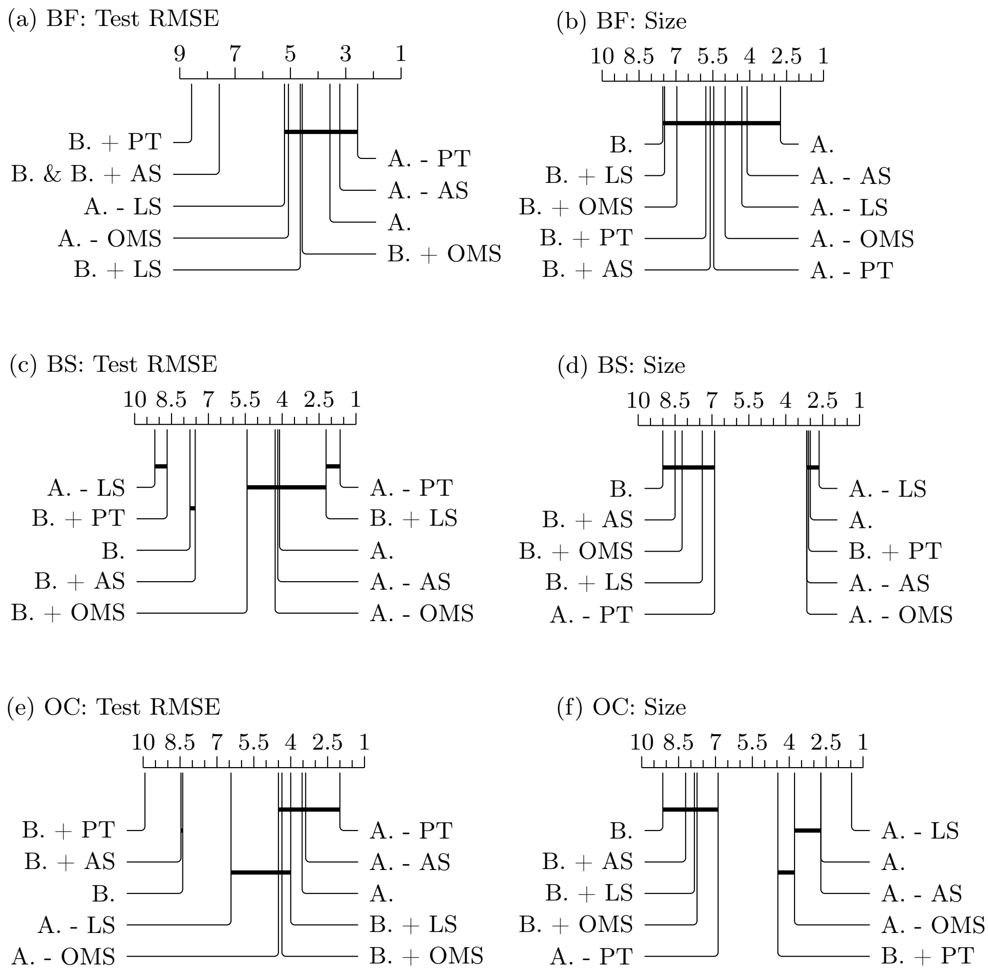


Fig. 3: Critical difference diagrams for the Best Fitness (BF), Optimal Compromise (OC) and Best Size (BS) models; where A.: ALL and B.: BASE.

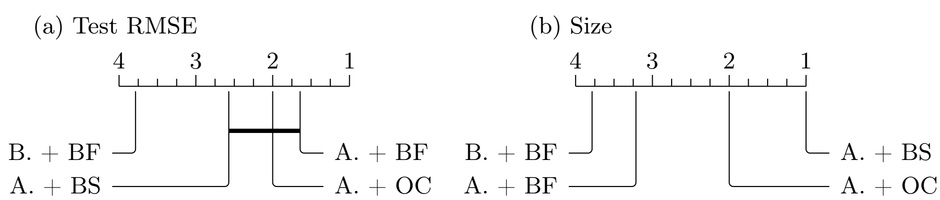


Fig. 4: Critical Difference Diagram comparing the final models (BF, BS and OC) found by the ALL variant with the SLIM (BASE) BF model.

Bibliography

[1] Martin Atzmueller, Johannes Fürnkranz, Tomáš Kliegr, and Ute Schmid. 2024. Explainable and interpretable machine learning and data mining. *Data Mining and Knowledge Discovery* 38, 5 (July 2024), 2571–2595. doi:10.1007/s10618-024-01041-y

[2] Illya Bakurov, José Manuel Muñoz Contreras, Mauro Castelli, Nuno Rodrigues, Sara Silva, Leonardo Trujillo, and Leonardo Vanneschi. 2024. Geometric semantic genetic programming with normalized and standardized random programs. *Genetic Programming and Evolvable Machines* 25, 1 (feb 2024), 29 pages. doi:10.1007/s10710-024-09479-1

[3] Bogdan Burlacu, Gabriel Kronberger, and Michael Kommenda. 2020. Operon C++: An Efficient Genetic Programming Framework for Symbolic Regression. In *Proceedings of the 2020 Genetic and Evolutionary Computation Conference Companion* (Cancún, Mexico) *(GECCO '20)*. Association for Computing Machinery, New York, NY, USA, 1562–1570.

[4] Mauro Castelli, Leonardo Trujillo, Leonardo Vanneschi, Sara Silva, Emigdio Z-Flores, and Pierrick Legrand. 2015. Geometric Semantic Genetic Programming with Local Search. In *Proceedings of the 2015 Annual Conference on Genetic and Evolutionary Computation* (Madrid, Spain) *(GECCO '15)*. Association for Computing Machinery, New York, NY, USA, 999–1006. doi:10.1145/2739480.2754795

[5] Miles Cranmer. 2023. Interpretable Machine Learning for Science with PySR and SymbolicRegression.jl. arXiv:2305.01582 [astro-ph.IM] https://arxiv.org/abs/2305.01582

[6] Edwin D. de Jong and Jordan B. Pollack. 2003. Multi-Objective Methods for Tree Size Control. *Genetic Programming and Evolvable Machines* 4, 3 (Sept. 2003), 211–233. doi:10.1023/a:1025122906870

[7] Davide Farinati, Illya Bakurov, and Leonardo Vanneschi. 2023. A study of dynamic populations in geometric semantic genetic programming. *Information Sciences* 648 (2023), 119513. doi:10.1016/j.ins.2023.119513

[8] Davide Farinati, Gloria Pietropolli, and Leonardo Vanneschi. 2025. A Study on the Dynamics and Effectiveness of the Deflate Geometric Semantic Mutation. *IEEE Transactions on Evolutionary Computation* (2025), 1–1. doi:10.1109/TEVC.2025.3611226

[9] Ivo Gonçalves, Sara Silva, and Carlos M. Fonseca. 2015. On the Generalization Ability of Geometric Semantic Genetic Programming. In *Genetic Programming*, Penousal Machado, Malcolm I. Heywood, James McDermott, Mauro Castelli, Pablo García-Sánchez, Paolo Burelli, Sebastian Risi, and Kevin Sim (Eds.). Springer International Publishing, Cham, 41–52.

[10] Maarten Keijzer. 2003. Improving Symbolic Regression with Interval Arithmetic and Linear Scaling. In *Genetic Programming*, Conor Ryan, Terence Soule, Maarten Keijzer, Edward Tsang, Riccardo Poli, and Ernesto Costa (Eds.). Springer Berlin Heidelberg, Berlin, Heidelberg, 70–82.

[11] Mark Kotanchek, Guido Smits, and Ekaterina Vladislavleva. 2007. *Pursuing the Pareto Paradigm: Tournaments, Algorithm Variations and Ordinal Optimization*. Springer US, 167–185. doi:10.1007/978-0-387-49650-4_11

[12] William La Cava, Patryk Orzechowski, Bogdan Burlacu, Fabricio de Franca, Marco Virgolin, Ying Jin, Michael Kommenda, and Jason Moore. 2021. Contemporary Symbolic Regression Methods and their Relative Performance. In *Proceedings of the Neural Information Processing Systems Track on Datasets and Benchmarks*, J. Vanschoren and S. Yeung (Eds.), Vol. 1. Curran. https://datasets-benchmarks-proceedings.neurips.cc/paper/2021/hash/c0c7c76d30bd3dcaefc96f40275bdc0a-Abstract-round1.html

[13] Si-Yang Liu, Hao-Run Cai, Qi-Le Zhou, and Han-Jia Ye. 2024. TALENT: A Tabular Analytics and Learning Toolbox. *arXiv preprint arXiv:2407.04057* (2024).

[14] James McDermott, Alexandros Agapitos, Anthony Brabazon, and Michael O'Neill. 2014. Geometric semantic genetic programming for financial data. In *Applications of Evolutionary Computation: 17th European Conference, EvoApplications 2014, Granada, Spain, April 23-25, 2014, Revised Selected Papers 17*. Springer, 215–226.

[15] Mohd Halim Mohd Noor and Ayokunle Olalekan Ige. 2025. A survey on state-of-the-art deep learning applications and challenges. *Engineering Applications of Artificial Intelligence* 159 (2025), 111225. doi:10.1016/j.engappai.2025.111225

[16] Alberto Moraglio, Krzysztof Krawiec, and ColinG. Johnson. 2012. Geometric Semantic Genetic Programming. In *Parallel Problem Solving from Nature - PPSN XII*, CarlosA.Coello Coello, Vincenzo Cutello, Kalyanmoy Deb, Stephanie Forrest, Giuseppe Nicosia, and Mario Pavone (Eds.). Lecture Notes in Computer Science, Vol. 7491. Springer Berlin Heidelberg, 21–31.

[17] Jose Manuel Muñoz Contreras, Leonardo Trujillo, Daniel E. Hernandez, and Luis A. Cardenas Florido. 2024. Benchmarking GSGP: Still competitive 10 years later? *Genetic Programming and Evolvable Machines* 26, 1 (Dec. 2024). doi:10.1007/s10710-024-09504-3

[18] Giorgia Nadizar, Fraser Garrow, Berfin Sakallioglu, Lorenzo Canonne, Sara Silva, and Leonardo Vanneschi. 2023. An Investigation of Geometric Semantic GP with Linear Scaling. In *Proceedings of the Genetic and Evolutionary Computation Conference* (Lisbon, Portugal) *(GECCO '23)*. Association for Computing Machinery, New York, NY, USA, 1165–1174. doi:10.1145/3583131.3590418

[19] Giorgia Nadizar, Berfin Sakallioglu, Fraser Garrow, Sara Silva, and Leonardo Vanneschi. 2024. Geometric semantic GP with linear scaling: Darwinian versus Lamarckian evolution. *Genetic Programming and Evolvable Machines* 25, 2 (2024), 17. doi:10.1007/s10710-024-09488-0

[20] Quang Uy Nguyen. 2011. *Examining Semantic Diversity and Semantic Locality of Operators in Genetic Programming*. Ph. D. Dissertation. Uni-

versity College Dublin, Ireland. http://ncra.ucd.ie/papers/Thesis_Uy_Corrected.pdf

[21] Peter Nordin and W. Banzhaf. 1995. Complexity Compression and Evolution. In *International Conference on Genetic Algorithms*. 310–317. https://api.semanticscholar.org/CorpusID:16415863

[22] Gloria Pietropolli, Davide Farinati, Luca Manzoni, Mauro Castelli, Sara Silva, and Leonardo Vanneschi. 2025. Introducing Crossover in SLIM-GSGP. In *European Conference on Genetic Programming (Part of EvoStar)*. Springer, 103–119.

[23] Gloria Pietropolli, Luca Manzoni, Alessia Paoletti, and Mauro Castelli. 2023. On the hybridization of geometric semantic GP with gradient-based optimizers. *Genetic Programming and Evolvable Machines* 24, 2 (Dec. 2023), Article number: 16. https://rdcu.be/dpWsR Online first.

[24] Liah Rosenfeld, Davide Farinati, Diogo Rasteiro, Gloria Pietropolli, Karina Brotto Rebuli, Sara Silva, and Leonardo Vanneschi. 2025. Slim_gsgp: A Python Library for Non-Bloating GSGP. In *Genetic and Evolutionary Computation Conference (GECCO '25)* (New York, NY, USA). ACM, Malaga, Spain, 9. doi:10.1145/3712256.3726398

[25] Ravid Shwartz-Ziv and Amitai Armon. 2022. Tabular data: Deep learning is not all you need. *Information Fusion* 81 (May 2022), 84–90. doi:10.1016/j.inffus.2021.11.011

[26] Guido F. Smits and Mark Kotanchek. 2005. *Pareto-Front Exploitation in Symbolic Regression*. Springer US, Boston, MA, 283–299. doi:10.1007/0-387-23254-0_17

[27] Leonardo Trujillo, Jose Manuel Muñoz Contreras, Daniel E. Hernandez, Mauro Castelli, and Juan J. Tapia. 2022. GSGP-CUDA — A CUDA framework for Geometric Semantic Genetic Programming. *SoftwareX* 18 (2022), 101085. doi:10.1016/j.softx.2022.101085

[28] Leonardo Vanneschi. 2024. SLIM_GSGP: The Non-bloating Geometric Semantic Genetic Programming. In *Genetic Programming*, Mario Giacobini, Bing Xue, and Luca Manzoni (Eds.). Springer Nature Switzerland, Cham, 125–141.

[29] Leonardo Vanneschi, Mauro Castelli, and Sara Silva. 2014. A survey of semantic methods in genetic programming. *Genetic Programming and Evolvable Machines* 15, 2 (June 2014), 195–214. doi:10.1007/s10710-013-9210-0

[30] Leonardo Vanneschi, Davide Farinati, Diogo Rasteiro, Liah Rosenfeld, Gloria Pietropolli, and Sara Silva. 2025. Exploring Non-bloating Geometric Semantic Genetic Programming. *Genetic Programming Theory and Practice XXI* (2025), 237–258.

[31] Marco Virgolin, T. Alderliesten, Cees Witteveen, and P. Bosman. 2021. Improving Model-Based Genetic Programming for Symbolic Regression of Small Expressions. *Evolutionary Computation* 29 (06 2021), 211–237. doi:10.1162/evco_a_00278